Proyecto final

Estafa del hijo accidentado: el AFD del Banco Nacional

10 % • escrito ≤ 10 pág. APA 7 • 10 diapositivas • **Python obligatorio** • plataforma UISIL + GitHub del curso

Este grupo

Grupo	6
Expo	viernes 4 de diciembre de 2026, 7:30 p.m.
Proyecto	Estafa del hijo accidentado: el AFD del Banco Nacional
Excel	`P06_Estafa_banco.xlsx`
Integrantes	Arias Céspedes Keylor Enrique Cubero Gamboa Gerson Adriano Villalobos Fernández Yeimy Priscila
Plataforma	UISIL (vía oficial: escrito y diapositivas)
GitHub	`PROYECTOS_MATEMATICA_DISCRETA_USIL_2026`

Se desarrolla en Python

El modelo, la lectura del Excel y los gráficos se programan en **Python 3.10+**. El entregable es una **aplicación** que lee el Excel y emite una decisión (no un cuaderno de ejercicios). El día de la exposición el grupo ejecuta `python main.py` en vivo. Si el programa no se ejecuta, esa parte vale cero. No se aceptan capturas en lugar de la ejecución.

Entrega

Vía oficial: plataforma UISIL (PDF del escrito y PDF de las diapositivas).
Además el código se publica en el repositorio del curso:
`github.com/Franciscocamposk/PROYECTOS_MATEMATICA_DISCRETA_USIL_2026`

- Hacer *fork*, colocar todo en `Jueves/Grupo_0X_Apellido/` o `Viernes/Grupo_0X_Apellido/` (`main.py`, Excel, `README.md`, `requirements.txt`, pruebas) y enviar un *pull request*.
- Escrito $\leq$ 10 páginas, APA 7, al menos 5 fuentes. Presentación $\leq$ 10 diapositivas; **todas** las personas hablan.
- El Excel del grupo (1000 a 1700 filas) se lee con `pandas`; no se copian valores.
- El `main.py` **genera un PNG** del modelo (grafo, Hasse, AFD o barras). Sin el gráfico en la ejecución, esa parte vale cero.
- Nota 10 %: escrito 4 + código 3 + exposición 3.

PROYECTO 6 | Viernes | LENGUAJES

Estafa del hijo accidentado: el AFD del Banco Nacional

6

Arias Céspedes Keylor Enrique • Cubero Gamboa Gerson Adriano • Villalobos Fernández Yeimy Priscila

AFD Lenguajes Estafa

Cliente. El filtro del Banco Nacional debe dejar pasar solo bncr.fi.cr y tirar el bit.ly del mensaje su-hijo-tuvo-un-accidente.

Problema. No basta un `re.match`: hay que definir δ y probar cadenas límite (https, dominio, @). El Excel trae mensajes marcados; el programa recalcula, no copia la marca.

Datos. Hoja `Mensajes`: url, correo, aceptado_oficial. Recalcular. Archivo: `P06_Estafa_banco.xlsx`. Carpeta en GitHub: `Viernes/Grupo_06_Apellido`.

Decisión. *¿El AFD acepta solo el enlace oficial del banco y rechaza la estafa del hijo?*

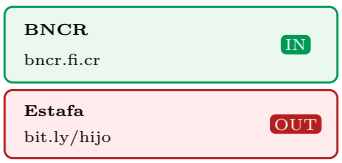


Ilustración del modelo.

Éxito. AFD dibujado por el código, 10 aceptadas, 10 rechazadas y coincidencia con el Python.

Trabajo requerido.

1. Documentar el formato (APA 7) y definir δ.
2. Implementar `acepta` sin regex opaco.
3. PNG del AFD (estados y transiciones).

Marco teórico. Lenguajes regulares, AFD. Hopcroft o Sipser cap. 1 (APA 7).

Gráfico obligatorio: lo genera el código

El `main.py` debe generar `afd_banco.png`: diagrama del AFD (nodos = estados, flechas = δ) y 4 cadenas de ejemplo pintadas. En la exposición el `main.py` debe crear el PNG (matplotlib o networkx). Sin el gráfico en la ejecución, esa parte vale cero.

Se desarrolla en Python

Tabla δ propia, `acepta(cadena)` y `afd_banco.png`. Validación: un enlace bncr.fi.cr pasa; bit.ly/hijo se rechaza. El `main.py` lee este Excel y se ejecuta en la exposición; imprime una decisión que el cliente entendería.